

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial



Grupo G-118

Integrantes:

- Mateo López Badias
- Martina Nudel
- Sofía Inés Ratto

Fecha de entrega: 14/06/2026

Tabla de Contenidos:

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1. Dominio.....	3
2.2. Lenguaje.....	4
3. Implementación.....	6
3.1. Frontend.....	6
3.2. Backend.....	7
3.2.1 Validación Semántica.....	7
3.2.2 Generación de Código.....	8
3.3. Dificultades Encontradas.....	8
4. Futuras Extensiones.....	9
5. Conclusiones.....	9
6. Bibliografía.....	10

1. Introducción

En este trabajo se desarrolló un lenguaje de dominio (DSL) orientado a la definición y validación de horarios académicos. El principal objetivo es permitir que el usuario pueda declarar aulas, docentes, comisiones, materias, disponibilidades y asignaciones a través de un lenguaje de texto simple, sin tener que encargarse manualmente de controlar todas las restricciones.

El compilador fue implementado en lenguaje C usando Flex para el análisis léxico y Bison para el análisis sintáctico. A partir del programa de entrada se crea una representación interna que después se analiza semánticamente. Durante esta etapa se chequea, entre otras cosas, que las entidades usadas hayan sido declaradas, que las aulas tengan capacidad suficiente, que no exista superposición de ningún tipo (ya sea docentes, aulas o comisiones), y que los horarios sean válidos.

Una vez hechas las validaciones, el compilador genera un archivo HTML con el horario resultante. Este archivo tiene las asignaciones de forma ordenada y se puede abrir directo en un navegador, sin necesidad de herramientas o librerías extra.

El desarrollo se realizó sobre el template Flex-Bison-Compiler provisto y se dividió en tres etapas: la definición del lenguaje y su dominio, la implementación del frontend léxico y sintáctico, y el desarrollo del backend, el cual incluye el análisis semántico y la creación del archivo de salida.

2. Modelo Computacional

Para diseñar SchedLang se partió de dos aspectos principales: el problema que se quería resolver y la forma de representarlo mediante un lenguaje. A partir de estas ideas se definieron las entidades que forman parte de un cronograma académico, las relaciones entre ellas y las condiciones necesarias para considerar válido un horario.

2.1. Dominio

El proyecto se enfoca en la creación y validación de cronogramas académicos. Organizar estos horarios requiere coordinar docentes, aulas, materias y comisiones, teniendo en cuenta la disponibilidad de cada recurso.

Cuando esta tarea se realiza manualmente, pueden aparecer errores como asignar un docente a dos clases al mismo tiempo, utilizar una misma aula para distintas comisiones en horarios superpuestos o elegir un aula sin capacidad suficiente. SchedLang busca detectar este tipo de problemas a partir de una descripción textual del cronograma.

El objetivo del lenguaje no es generar automáticamente el mejor horario posible, sino permitir que el usuario proponga uno y verificar si cumple las reglas establecidas.

El dominio incluye las siguientes entidades:

- Docente: persona encargada de dictar una comisión
- Aula: espacio donde se desarrolla una clase. Cada aula tiene una capacidad máxima
- Materia: unidad académica que incluye una cantidad de estudiantes
- Comision: instancia concreta de una materia
- Disponibilidad: horario en el que un docente puede ser asignado
- Bloqueo: periodo en el que un docente no puede escribir asignaciones
- Asignación: relación entre una comisión, un docente, un aula y un horario

En la versión actual, cada programa describe un sólo cronograma. Las entidades deben dictarse antes de ser utilizadas. Por ejemplo, una materia debe existir antes de crear una comisión asociada, y un docente debe declararse antes de indicar sus horarios disponibles.

Durante la validación se comprueba que los nombres no estén repetidos, que las capacidades y cantidades de estudiantes sean positivas y que todas las referencias correspondan a entidades existentes. También se verifica que el aula tenga espacio suficiente para los estudiantes de la materia.

En cuanto a los horarios, una asignación debe estar completamente incluida dentro de la disponibilidad del docente y no debe coincidir con un horario bloqueado. Además, un docente, un aula o una comisión no pueden formar parte de dos asignaciones superpuestas.

Las franjas contiguas sí están permitidas. Por ejemplo, una clase puede finalizar a las 10:00 y la siguiente comenzar en ese mismo momento. También pueden existir clases simultáneas cuando utilizan docentes, aulas y comisiones diferentes.

2.2. Lenguaje

SchedLang utiliza una sintaxis declarativa y lineal. Cada sentencia representa una acción del cronograma y termina con un punto y coma. Los nombres se escriben entre comillas, mientras que los días de la semana se expresan en inglés y en minúsculas.

Construcción	Descripción
teacher	Declara un docente
room	Declara un aula y su capacidad
course	Declara una materia y su cantidad de estudiantes
section	Declara una comisión asociada a una materia

available	Indica la disponibilidad de un docente
blocked	Indica un horario bloqueado para un docente
assign	Asigna una comisión, un docente y un aula a un horario
print schedule	Solicita la generación del cronograma en HTML

Tabla 1: Construcciones principales de SchedLang.

Las entidades básicas se declaran de la siguiente manera:

```
teacher "Perez";
room "A101" capacity 40;
course "Autómatas" students 35;
section "Automatas-1" of "Autómatas";
```

En este ejemplo se declara un docente, un aula con capacidad para cuarenta personas, una materia con treinta y cinco estudiantes y una comisión asociada a esa materia.

Luego se pueden definir los horarios disponibles y bloqueados de los docentes:

```
available "Perez" monday 08:00-12:00;
blocked "Perez" monday 10:00-11:00;
```

La primera sentencia indica que el docente está disponible el lunes entre las 08:00 y las 12:00. La segunda establece que, dentro de ese período, no puede ser asignado entre las 10:00 y las 11:00.

Las asignaciones se expresan vinculando las entidades declaradas:

```
assign "Automatas-1" to "Perez" in "A101" at monday 08:00-10:00;
```

Esta sentencia indica que la comisión Automatas-1 será dictada por Perez en el aula A101, el lunes de 08:00 a 10:00.

Un programa completo puede verse de la siguiente forma:

```
teacher "Perez";

room "A101" capacity 40;

course "Autómatas" students 35;
section "Automatas-1" of "Autómatas";

available "Perez" monday 08:00-12:00;

assign "Automatas-1" to "Perez" in "A101" at monday 08:00-10:00;

print schedule;
```

Código 1: Ejemplo de un programa válido en SchedLang.

Las palabras utilizadas por el lenguaje se eligieron buscando que cada sentencia se parezca a la acción que representa. Esto permite entender un programa sin necesidad de conocer cómo funciona internamente el compilador.

SchedLang permite declarar múltiples docentes, aulas, materias y comisiones, definir varias disponibilidades para un mismo docente y realizar asignaciones en distintos días y horarios. Durante la compilación se validan las referencias, la capacidad de las aulas, la disponibilidad de los docentes y las posibles superposiciones.

Cuando aparece la sentencia `print schedule`, se genera el archivo `schedule.html`. Este contiene un resumen del cronograma y una tabla con la materia, la comisión, el docente, el aula, el día, el horario y la ocupación correspondiente. Las asignaciones se muestran ordenadas por día y hora para facilitar su lectura.

3. Implementación

Para la implementación se usó como base el proyecto Flex-Bison-Compiler provisto. A partir del mismo se desarrolló el frontend con Flex y Bison, y después, se incorporó el backend que se encarga del análisis semántico y de generar el archivo HTML.

3.1. Frontend

En el frontend se definieron los elementos necesarios para poder representar un horario académico. Entre los símbolos terminales están las palabras reservadas **teacher**, **room**, **course**, **section**, **available**, **blocked**, **assign** y **print**, además de los días de la semana, números enteros, cadenas de texto y horarios.

A partir de estos símbolos se definieron las producciones gramaticales que corresponden con las distintas sentencias del lenguaje:

- Declaración de docentes
- Declaración de aulas y sus capacidades
- Declaración de materias y cantidad de estudiantes
- Declaración de comisiones asociadas a una materia

- Disponibilidad y bloqueo de docentes
- Asignación de una comisión a un docente, aula, día y horario
- Impresión del horario creado

El analizador léxico reconoce las palabras reservadas, cadenas, valores numéricos, horarios y días. A su vez, el mismo descarta espacios y comentarios que no afectan al significado del programa. Al encontrar un elemento válido, devuelve el token correspondiente al parsear.

El analizador sintáctico chequea que las sentencias representen la estructura definida por la gramática. Por ejemplo, controla que una declaración tenga todos sus componentes en el orden correcto y que finalice con punto y coma. En esta etapa también se crea el AST, en donde cada sentencia queda representada por un nodo con la información que se necesita para las etapas posteriores.

Algunas de las restricciones no se pueden comprobar solo desde la gramática. Estas se verifican posteriormente en el backend.

3.2. Backend

El backend se ocupa de realizar las validaciones que no pueden resolverse sólo con la gramática, en especial las que dependen de las relaciones entre docentes, aulas, horarios, comisiones y materias principales incluyen:

- Las entidades usadas tienen que haber sido declaradas previamente.
- No pueden haber docentes, aulas, comisiones o materias duplicadas.
- La capacidad del aula tiene que ser suficiente para la cantidad de alumnos.
- Los horarios tienen que tener una hora de inicio anterior a la de finalización.
- Una asignación tiene que estar incluida en la disponibilidad del docente.
- Una asignación no puede superponerse con un bloqueo.
- Un docente no puede estar en dos aulas al mismo tiempo.
- Un aula no puede tener dos comisiones en horarios superpuestos.
- Una comisión no puede tener dos asignaciones simultáneas.

Una vez que el programa se valida, el backend genera un archivo HTML que tiene el horario resultante.

3.2.1 Validación Semántica

El analizador semántico (SemanticAnalyzer.c) recorre el AST y construye un modelo intermedio a través del módulo ScheduleModel. En este modelo se guardan los docentes, aulas, materias, comisiones, disponibilidades, bloqueos y asignaciones.

Durante el recorrido se resuelven las referencias entre las distintas entidades. Por ejemplo, una comisión se vincula con su materia y una asignación se vincula con su comisión, docente y aula. Si alguna referencia no existe, el programa es rechazado.

Para validar los conflictos se comparan los días y los intervalos de horarios de las asignaciones. Dos intervalos se consideran superpuestos cuando comparten parte de su duración. Sin embargo, se pueden tener horarios contiguos, por lo que una clase puede comenzar justo cuando termina la anterior.

También se chequea que la asignación se encuentre completamente dentro de alguna de las ventanas de disponibilidad del docente y que no coincida con ninguno de sus bloqueos. Si se encuentra un error, se avisa el por qué y el compilador finaliza con un estado de error sin generar el archivo de salida.

3.2.2 Generación de Código

El generador (CodeGenerator.c) se ejecuta después de que el análisis semántico termine correctamente. Cuando el programa contiene la sentencia `print schedule`, genera o reemplaza automáticamente el archivo `schedule.html`.

El archivo generado tiene las asignaciones a través de una tabla que incluye:

- Materia y comisión.
- Aula y docente.
- Día y horario.
- Cantidad de alumnos y capacidad del aula.

También hay un resumen con la cantidad de asignaciones, docentes y aulas utilizadas. Si el programa no tiene ninguna asignación, se genera igualmente un HTML válido con un mensaje indicando que el horario está vacío.

Durante la creación se escapan los caracteres especiales de HTML, como `&`, `<`, `>`, comillas simples y dobles. Esto nos deja usar esos caracteres en los nombres sin cambiar accidentalmente la estructura del documento.

El archivo incluye sus propios estilos CSS y no depende de librerías externas. Por este motivo, el resultado se puede abrir directamente en cualquier navegador sin necesitar un runtime adicional.

3.3. Dificultades Encontradas

Una de las dificultades más grandes fue separar las validaciones sintácticas de las semánticas. La gramática nos deja reconocer la estructura de una asignación, pero no puede determinar si las entidades mencionadas existen o si se produce un conflicto con otras asignaciones. Esto se resolvió construyendo un modelo semántico separado del AST.

Otra dificultad fue la comparación de los intervalos horarios. Era necesario detectar correctamente las superposiciones sin rechazar horarios consecutivos. Para esto, los intervalos se consideraron semiabiertos, permitiendo que uno comience en el mismo momento en el que finaliza otro.

Para finalizar, se debió controlar la generación del archivo HTML entre las distintas pruebas. Para evitar que un archivo generado en una ejecución anterior genere resultados incorrectos, el script elimina `schedule.html` antes y después de cada caso. Además, se agregaron pruebas específicas para comprobar el contenido generado y el escape de caracteres especiales.

4. Futuras Extensiones

El mecanismo de validación de salida HTML actual verifica la presencia de fragmentos individuales mediante `grep --fixed-strings`, lo que resulta insuficiente para garantizar la corrección del ordenamiento producido por el generador de código. En particular, la función `qsort` aplicada sobre las asignaciones —ordenadas por día y luego por hora de inicio— no es ejercida por los tests actuales: un generador que emita las filas en orden inverso o arbitrario pasaría igualmente la suite. Una extensión natural sería reemplazar la comparación por fragmentos con una verificación posicional, ya sea extrayendo el contenido de las filas con `grep -n` y comparando números de línea, o bien generando un archivo de salida de referencia completo y realizando un diff exacto contra el HTML producido. Esta segunda alternativa nos dejaría además detectar duplicados, contenido ilegítimo y cualquier regresión estructural en la plantilla HTML, a costa de requerir que los archivos de referencia se actualicen manualmente cada vez que se modifique el template del generador.

También se podría incorporar un número de legajo como identificador único para docentes. Actualmente, los docentes se identifican a través de su nombre, lo que puede generar ambigüedades si existen dos personas con el mismo nombre. El uso del legajo nos dejaría diferenciar de manera inequívoca y tener el nombre solo como información descriptiva. Para implementar esto, habría que cambiar la gramática, el AST y el modelo semántico, y adaptar las validaciones y la creación del archivo HTML.

Otra posible extensión consiste en incorporar algunas funcionalidades planteadas durante el Stage I que no fueron incluidas en el alcance final de la implementación. Entre ellas se encuentran la definición de múltiples horarios nombrados e independientes, la clasificación de aulas según su tipo o equipamiento, la posibilidad de establecer un máximo diario de asignaciones por docente, la carga horaria requerida por cada materia y la declaración de franjas horarias como entidades independientes. Estas funcionalidades permitirían modelar situaciones académicas más complejas, aunque requerirían extender la gramática, el AST, el modelo semántico, las validaciones y la generación del archivo HTML.

5. Conclusiones

En conclusión, se logró desarrollar un compilador funcional para SchedLang, un lenguaje declarativo orientado a la creación y validación de cronogramas académicos. Se implementó un frontend con Flex y Bison, junto con un backend capaz de detectar conflictos de horarios, validar referencias, disponibilidades y capacidades, y generar un archivo HTML con el cronograma resultante.

A pesar de las dificultades encontradas durante el desarrollo, se cumplió con el objetivo principal y se obtuvo una base clara y funcional sobre la cual podrían incorporarse nuevas características en el futuro.

6. Bibliografía

1. (2026-03-12) Proyecto Especial
<https://docs.google.com/document/d/1XZy63UwUVHyMCR6p051aer-i9SpbLmoOSGC-gh6Cb-I/edit?usp=sharing>
2. (2024-09-25, v0.2.0) Análisis Semántico
https://docs.google.com/document/d/1ZqUt72POtTA-IXPHUVgFqz0hbb7-QQnc5i_Q0B1rMPY/edit?usp=drive_link
3. (2024-05-16, v0.1.0) Tabla de Símbolos
https://docs.google.com/document/d/15-LxLzKMRrU95Jx1F_mttYzP4VmYq4XWHrBTl2hzWPo/edit?usp=drive_link
4. (2024-05-16, v0.1.0) Scopes
https://docs.google.com/document/d/1AT9Yw1as6Nq7REIcixjX87jUB6BX679tEUU7ugLiGIM/edit?usp=drive_link
5. (2024-05-16, v0.1.0) Sistema de Tipos
https://docs.google.com/document/d/18dNU_QyZ6M36lj72ivm5Dqsm4KZzVjfd2MqmYeuIz3M/edit?usp=drive_link
6. (2024-05-16, v0.1.0) Generación de Código
https://docs.google.com/document/d/1gTDatettvr3GLewUy4_iwacQvTCZAAks02xu6l6eFcc/edit?usp=drive_link
7. (2024-05-16, v0.1.0) Runtime
https://docs.google.com/document/d/1prPD3SI4nWVyIHlmTplm4AN-szqARGIkOKx-xEq6uv0/edit?usp=drive_link